TRANSFORMACIÓN DE CLAVES

HASHING DISPERSIÓN

ESTRUCTURAS DE DATOS
y ALGORITMOS
LCC – LSI - TUPW

Objetivos

- Conocer la Técnica de Hashing.
- Evaluar el costo computacional de las operaciones, para las distintas políticas de manejo de colisiones.
- Construir el TAD Tabla Hash.

HASHING

Problema

Almacenar y recuperar un elemento particular, identificado por medio de su valor de clave, con el **menor *costo* posible**.

HASHING

Buscar - Insertar

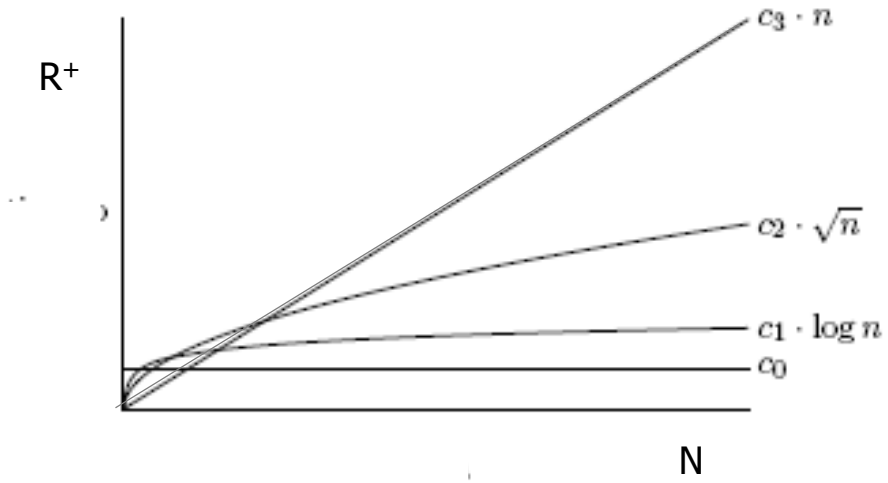
ABB

Lista

Árbol B

AVL

Lista ordenada por contenido



Hashing

HASHING

Los datos se almacenan en direcciones de memoria, entonces:

¿Cómo determinar la dirección de memoria en la cual almacenar cada elemento?



TABLAS DE ACCESO DIRECTO

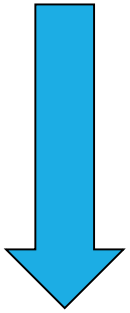
TABLAS HASH

HASHING

TABLAS HASH

Almacenamiento y Recuperación

H es la transformación de claves –**K**- en direcciones –**D**-



$$H : K \rightarrow D$$

T.D.A. HASHING

Si K = { ki / 000 <= ki <= 999}

y |K| = 1000



D = { di / 000 <= di <= 999}

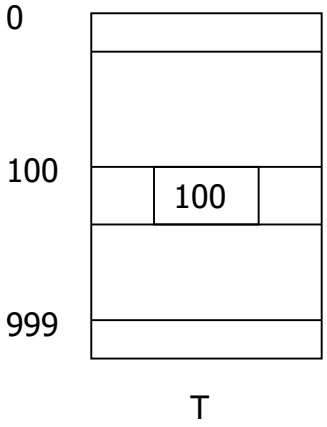
y |D| = 1000

¿Cuál es la función H: K→D ?

Identidad : K → D, esto es:

$h(k_i) = id(k_i) = d$

Si k = 100 entonces id(100) = 100 = d



Si tenemos N claves, con valores mayores a N?

HASHING
Funciones biyectivas

En matemáticas, una función

F : X → Y

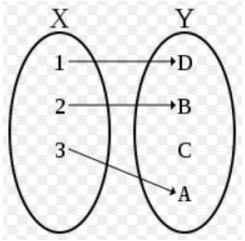
es **biyectiva** si es al mismo tiempo **inyectiva** y **sobreyectiva**

F es **inyectiva** si a elementos distintos del conjunto X (dominio) les corresponden elementos distintos en el conjunto Y (codominio)

$$\forall a, b \in X,\ f(a) = f(b) \Rightarrow a = b$$

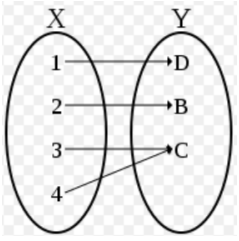
que es equivalente a su contrarrecíproco

$$\forall a, b \in X,\ a \neq b \Rightarrow f(a) \neq f(b)$$[1]



F es **sobreyectiva** si está aplicada sobre todo el codominio, es decir, cuando cada elemento de Y es la imagen de como mínimo un elemento de X

$$\forall y \in Y \quad \exists x \in X : \quad f(x) = y$$



HASHING

Funciones biyectivas

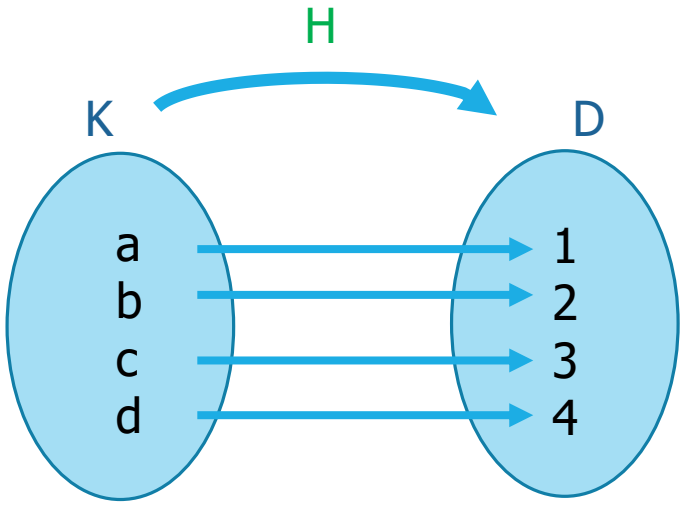
F : X → Y

- Inyectiva
- Sobreyectiva

H : K→ D

¿Inyectiva?

¿Sobreyectiva?



HASHING PERFECTO

HASHING PERFECTO

- Todas las posiciones de la tabla T están ocupadas
- Para dos claves distintas k1 y k2, se cumple que h(k1) es distinta de h(k2).

$$k1 \neq k2 \quad \wedge \quad h(k1) \neq h(k2)$$

HASHING PERFECTO

El rango de valores posibles de las claves **K** (Ejemplo DNI)
Generalmente es mayor que el tamaño de **D** (arreglo)

$$H: K \longrightarrow D$$

$$\mathbf{k1 \neq k2} \quad y \quad \mathbf{h\,(k1) = h\,(k2) = d}$$

k1 y k2 son claves **sinónimas**

que **colisionan** en la dirección **d** bajo la transformación **h.**

HASHING

Aspectos Relevantes

a) Elección de la función de transformación H.

- Distribuir las claves uniformemente
- Ser calculable de modo eficiente

Método de la División

Extracción

Plegado

Cuadrado Medio

Funciones aplicables a claves alfanuméricas

b) Política de manejo de colisiones.

Hashing Abierto

Encadenamiento

Hashing Cerrado.

Uso de Buckets o cubos

Direccionamiento abierto.

HASHING
Funciones de transformación

Método de la División

$h(k) = k \bmod M$

Extracción

Extraer de la clave, los dígitos que varían mas aleatoriamente

Plegado

Si $k_i = k_{i1}\ k_{i2} \mid \ldots \mid k_{in-1}\ k_{in}$

$$h(k_i) = k_{i1}\ k_{i2} + \ldots + k_{in-1}\ k_{in}$$

HASHING
Funciones de transformación (2)

Cuadrado Medio

$k_i = k_{i1}\ k_{i2}\ \ldots\ k_{in-1}\ k_{in}$

$h(k_i) = k_i^{\,2}$ y luego se extraen los dígitos centrales

$d_i = k_{ij}\ k_{ij+1}\ \ldots.\ k_{ij+l}$ $\quad 1 < j < j+l < n$ y $\quad 0 <= kij\ kij+1\ \ldots.\ kij+l <= M-1$

Funciones aplicables a claves alfanuméricas

$ki = c_{i1}\ c_{i2}\ c_{i3}\ \ldots.\ c_{in}$

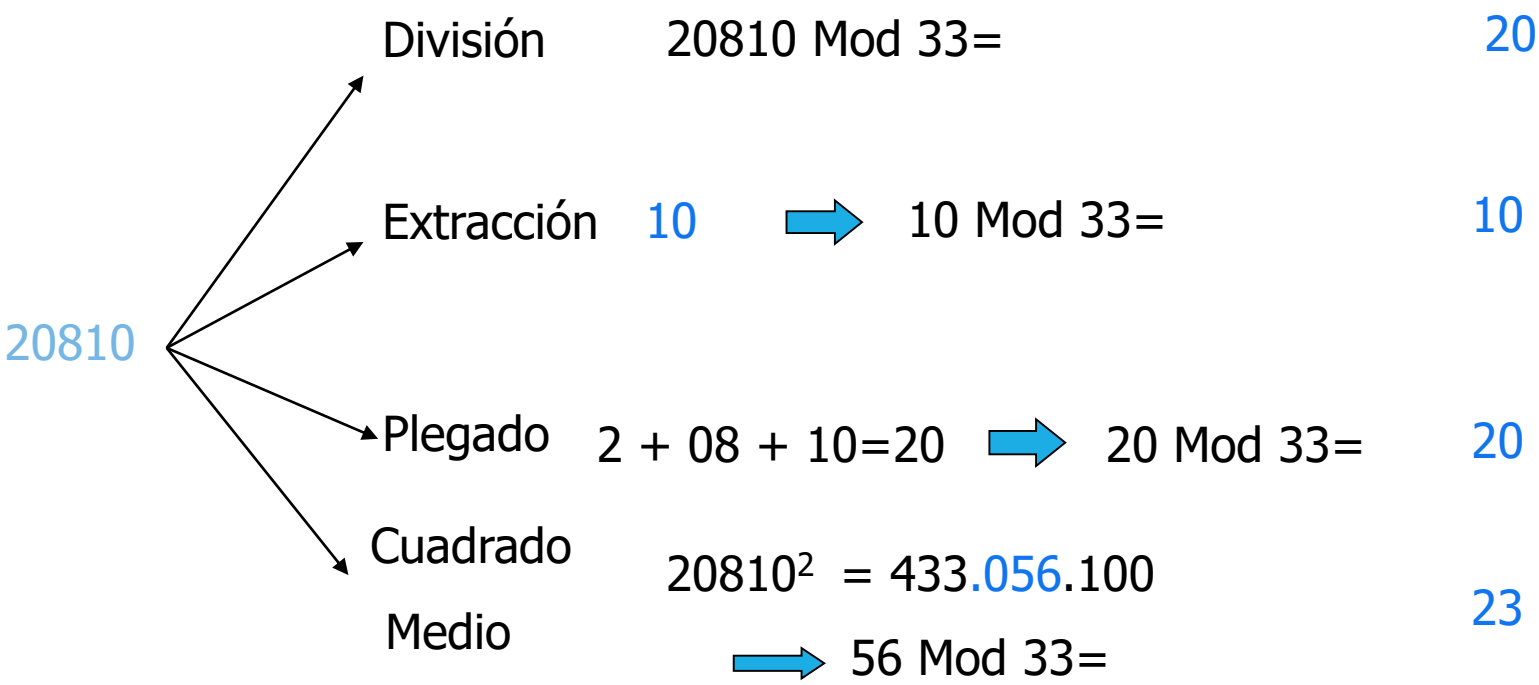
$h(k_i) = ASCII(c_{i1}) + ASCII(c_{i2}) + \ldots\ldots + ASCII(c_{in})$

$h(k_i) = ASCII(c_{i1}) * b^1 + ASCII(c_{i2}) * b^2 + \ldots\ldots + ASCII(c_{in}) * b^n$

HASHING
Funciones de transformación (3)

Claves = {20810, 21438, 21478, 21755, 21705, 21762, 21444,…}

$|\text{Claves}| = \mathbf{33}$



HASHING

Funciones de transformación (4)

CLAVES	MOD 33		PLEGADO	CUADRADO MEDIO
20810	20	10	20	23
21438	21	5	21	26
21478	28	12	28	7
21755	8	22	8	16
21705	24	5	24	8
21762	15	29	15	23
21444	27	11	27	20

HASHING
Funciones de transformación (6)

$h1(k_i) = ASCII(c_{i1}) + ASCII(c_{i2}) + \ldots\ldots + ASCII(c_{in})$

k_1 : ROMA k_2 : RAMO

Caracter	ASCII
R	82
O	79
M	77
A	65

h1 (ROMA) = 82 + 79 + 77 + 65 =303 = 82 + 65 + 77 + 79 = h1(RAMO)

$h2(k_i) = ASCII(c_{i1}) * b^1 + ASCII(c_{i2}) * b^2 + \ldots\ldots + ASCII(c_{in}) * b^n$

h2 (ROMA) = $82 * 10^1 + 79 * 10^2 + 77 * 10^3 + 65 * 10^4$ = 735720

h2 (RAMO) = $82 * 10^1 + 65 * 10^2 + 77 * 10^3 + 79 * 10^4$ = 163320

HASHING

Aspectos Relevantes

a) Elección de la función de transformación H.

- distribuir las claves uniformemente
- ser calculable de modo eficiente

Método de la División

Extracción

Plegado

Cuadrado Medio

Funciones aplicables a claves alfanuméricas

b) Política de manejo de colisiones.

Hashing Abierto

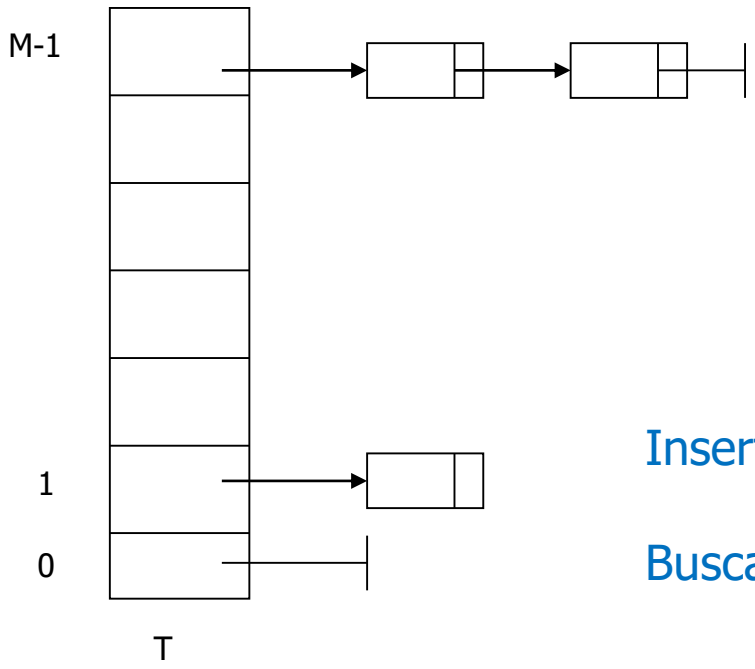
- *Encadenamiento*

Hashing Cerrado.

- *Uso de Buckets o cubos*
- *Direccionamiento abierto.*

HASHING
Políticas de manejo de colisiones (1)

Hashing Abierto :Encadenamiento



¿ M ?

Calculo de antemano cuantas colisiones estoy dispuesto a aceptar

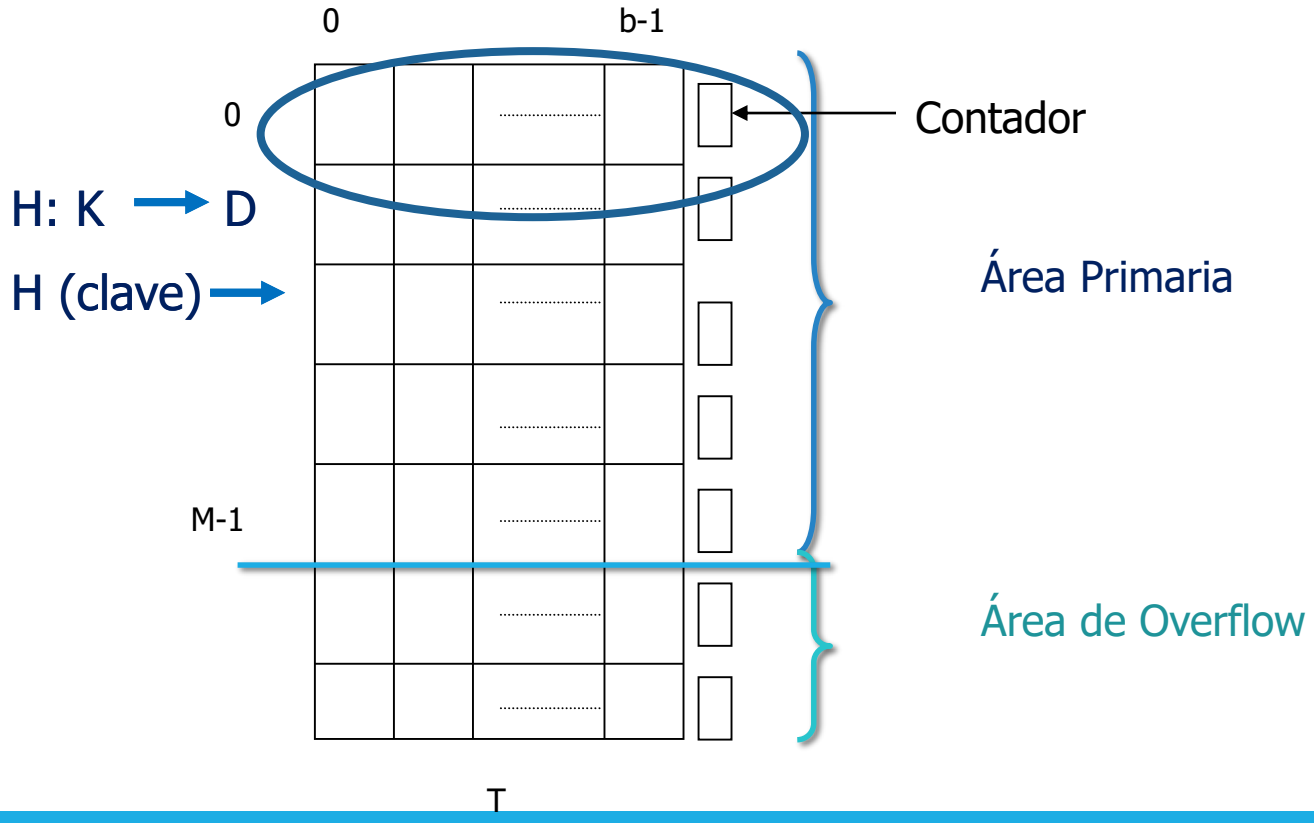
M=N/ cant_colisión

Insertar cabeza de la lista (Pila) → O(1)

Buscar Si está vacío → no existe
sino lo busco en la lista → O(n)

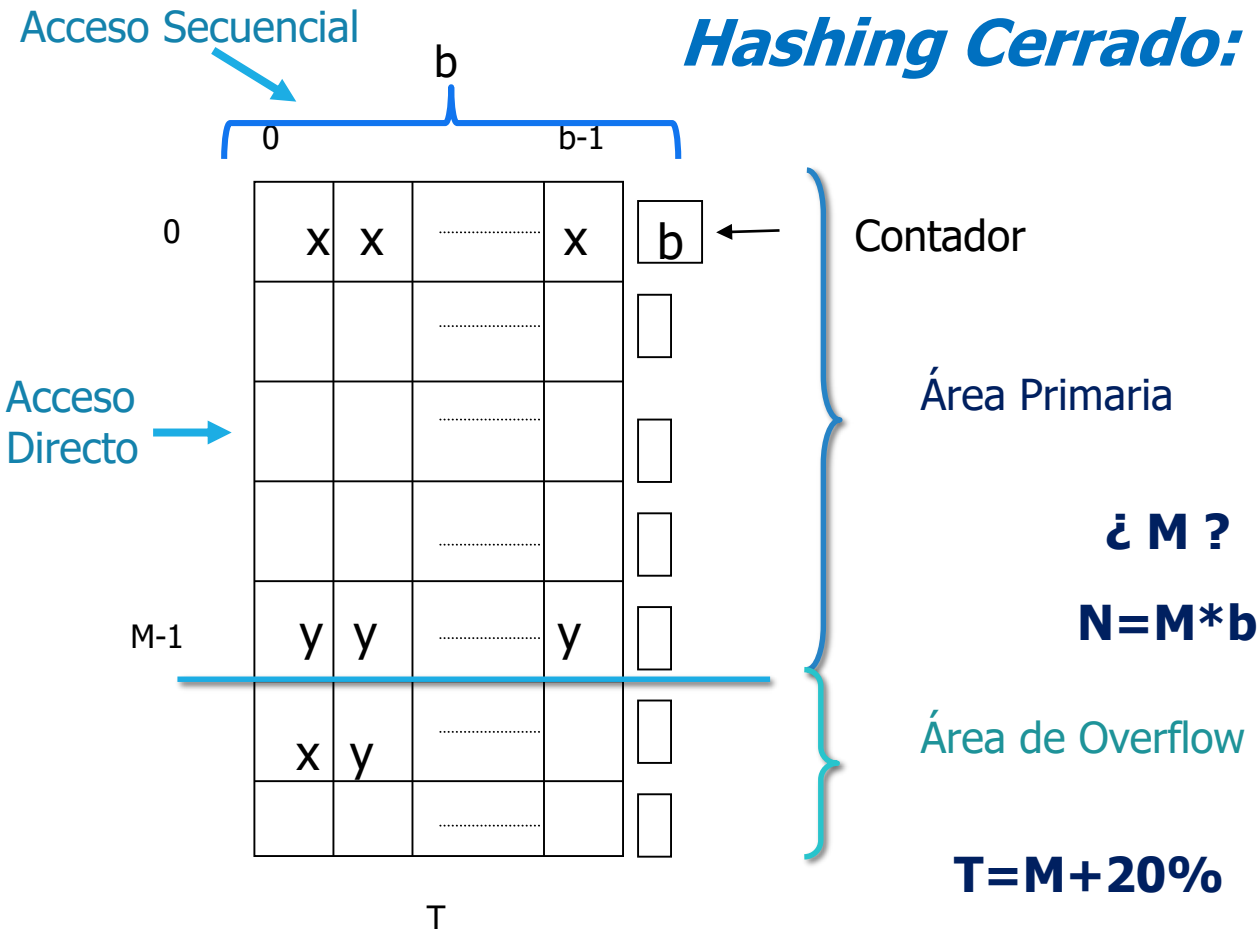
HASHING
Políticas de manejo de colisiones (2)

Hashing Cerrado:Uso de Buckets o Cubos



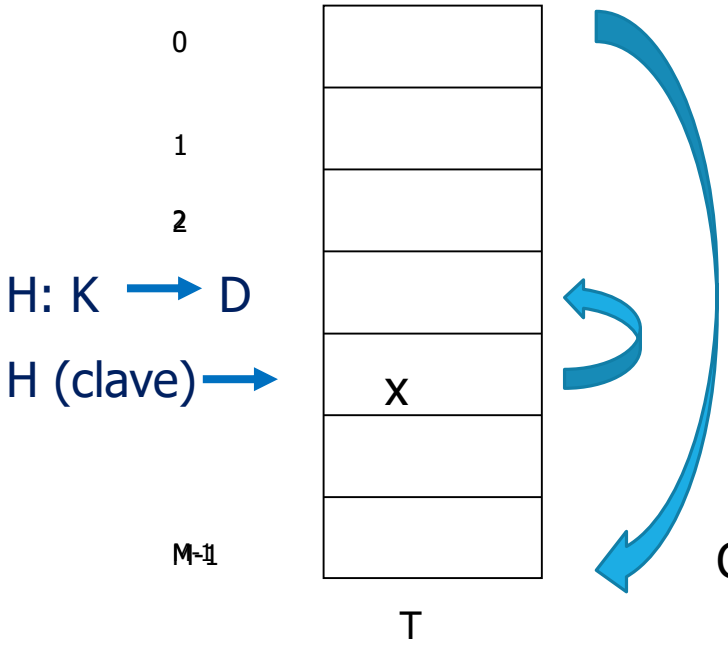
HASHING
Políticas de manejo de colisiones (3)

Hashing Cerrado: Uso de Buckets



HASHING
Políticas de manejo de colisiones (4)

Hashing Cerrado : Direccionamiento Abierto



Circular

Insertar

Si esta ocupado → D+1 o D-1

Buscar

Si no es lo que busco → D+1 o D-1

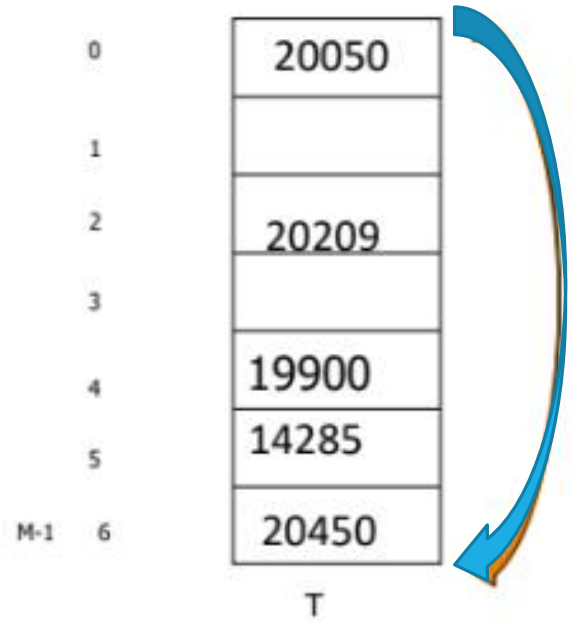
Utilizo siempre la misma política

Ver Video Direccionamiento Abierto.mp4

HASHING
Políticas de manejo de colisiones (4)

Hashing Cerrado : Direccionamiento Abierto

k=19900
H(k)=0



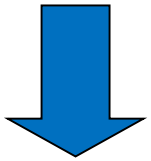
Claves	Mod 7	Extracción	Plegado	Cuadrado Medio
20050	2	0	0	1
14285	5	5	6	6
20450	3	0	4	0
19900		0		
18364		4		
20209	0	2	6	0
20182		2		

HASHING
Políticas de manejo de colisiones

- *Secuencia de prueba lineal*

h(k), h(k)-1,.....,1, 0, M-1, M-2,....., h(k)+1
reducida a módulo tamaño de la tabla

Agrupamientos Primarios!!!

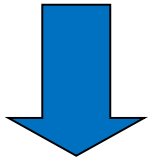


- *Secuencia de prueba pseudo random*

h(k)+0, h(k)+r1, ..., h(k)+rm-1
reducida a módulo tamaño de la tabla

En lugar de r1…rm-1, se puede usar un solo nº aleatorio para toda la secuencia

Agrupamientos Secundarios!!!



- *Doble Hashing*

*h(k) – i * h2 (k) para 0 <= i <= M-1*

HASHING *Factor de Carga*

Standish define al ***factor de carga*** α de una tabla como la razón entre el número de entradas N ocupadas en la tabla y el número total de entradas M en la tabla

$$\alpha = N/M$$

Este autor expresa que si α **<= 0.7**, la cantidad de comparaciones involucradas en la recuperación de un elemento es adecuada.

Esta propuesta surge al considerar que la performance se deteriora especialmente al aplicar la secuencia de prueba lineal, a medida que la tabla se llena – cuando N se aproxima a M **=>** Complejidad O(N)

HASHING *Factor de Carga*

N=33

M=33

$\Rightarrow$ **α = N/M = 1**
Complejidad O(N)



α = 0.7 y 0.7= N/M
=>M=N/0.7
=>M=33 / 0.7 => 47?

Nros primos

2, 3, 5, 7, 11, 13, 17,
19, 23, 29, 31, 37, 41,
43, **47**, 53, 59, 61, 67,
71, 73, 79, 83, 89 y 97

T

XXXX	0
XXXX	1
XXXX	
	7
21755	8
XXXX	9
	19
20810	20
21438	21
XXXX	22
XXXX	
20551	30
XXXX	
XXXX	M-1

T.A.D. Tabla HASHING

Operaciones Abstractas

Crear

Insertar

Buscar

Se pretende generar un TAD con una complejidad temporal de O(1) en el acceso a los datos: Insertar y Buscar
(otras operaciones pueden ser de O(n)).